

# Schedulers and Performance — A Readable Companion to the OTP Docs

Chapter 10 · checked against OTP 29.1.1 · Sep 25, 2026

## 1. Why BEAM performance is about latency first

The BEAM is built to keep many things responsive at once, not to finish one thing as fast as possible. Its schedulers interrupt every process after a small budget of work, so one busy process cannot freeze the others. That fairness is why a node with a million processes still answers promptly, and it shapes what “fast” means here.

Most real performance problems on the BEAM are not slow arithmetic. They are a single process everyone waits on, a mailbox that keeps growing, a native call that won’t give the CPU back, or memory growth that makes garbage collection expensive. The tools and habits in this chapter aim at those.

## 2. The building blocks

### Schedulers

By default the runtime starts one normal scheduler thread per CPU core. Each has its own run queue and moves Erlang processes on and off the core. A process runs until it uses up its budget of *reductions* (roughly, function calls and units of built-in work), blocks in `receive`, or finishes; then the next process gets a turn. This is preemptive multitasking in user space, and it’s why an infinite loop in one process does not stop the others.

Two extra pools exist for work that can’t be interrupted:

| Pool                 | For                      | Default size |
|----------------------|--------------------------|--------------|
| Normal schedulers    | All Erlang code          | One per core |
| Dirty CPU schedulers | Long native CPU work     | One per core |
| Dirty I/O schedulers | Long native blocking I/O | 10           |

### Profiling tools

The docs are direct: “Even experienced software developers often guess wrong about where the performance bottlenecks are.” Measure first.

| Tool                                | Measures                                           | Overhead      |
|-------------------------------------|----------------------------------------------------|---------------|
| <code>tprof</code>                  | Call counts, time, or heap allocation per function | Moderate      |
| <code>eprof</code>                  | Time per function, per process                     | Small         |
| <code>cprof</code>                  | Call counts only                                   | Low           |
| <code>fprof</code>                  | Full call-graph timing                             | Large         |
| <code>perf</code> (Linux, with JIT) | Sampled native profile                             | Much lower    |
| <code>lcnt</code>                   | Lock contention inside the runtime                 | Special build |

## 3. How the runtime actually does it

---

### Where parallelism comes from

Only separate processes run in parallel. One process runs on one scheduler at a time, so all work done inside a single `gen_server` is sequential, however many cores you have. Scaling on the BEAM means splitting work across processes: per request, per key, per partition.

### Native code and the 1 ms rule

A NIF runs inside a scheduler thread and cannot be preempted. The docs' guideline is that a well-behaved NIF returns "within 1 millisecond." Longer calls block that scheduler and every process queued on it, and the docs list the effects: "degraded responsiveness," "extreme memory usage, and bad load balancing between schedulers." And "a native function that crashes will crash the whole VM."

The fixes, from simplest to most work:

- mark the NIF dirty (CPU-bound or I/O-bound) so it runs on a dirty scheduler
- split the work into chunks with `enif_schedule_nif`
- report progress with `enif_consume_timeslice` and yield when told to

Classifying CPU-bound work as I/O-bound can starve the normal schedulers, the docs warn.

### Benchmarking honestly

The docs recommend making each measurement last several seconds, running each test in a fresh process (reused processes start with larger heaps), and preferring `statistics(runtime)` (CPU time) over `timer:tc` (wall-clock time) when results vary. They also warn that the fastest implementation on one machine is not always the fastest on another.

## 4. Plausible but wrong

### 4.1 Benchmarking in the shell

```
1> timer:tc(fun() -> lists:foldl(fun(X, A) -> X + A end, 0, lists:seq(1, 1000000)) end).
```

Funs typed in the shell are interpreted by `erl_eval`, not compiled, so the number is many times slower than the real code. It is also a single short run in a shell process with an already grown heap. Fix: put the code in a compiled module, run it for seconds in a fresh process, and compare alternatives the same way, or use a benchmarking tool such as `erlperf`.

### 4.2 Optimising by guessing

```
%% "lists:map is slow, write it by hand"
double([]) -> [];
double([H | T]) -> [H * 2 | double(T)].
```

This is the kind of change an assistant suggests readily. It gains little or nothing, and the real cost is usually elsewhere: a `++` in a loop (chapter 3), a table scan (chapter 4), or a process everybody waits on. Fix: profile first with `tprof` or `perf`, and optimise the functions with the highest *own* time.

### 4.3 One process doing all the CPU work

```
handle_call({resize, Img}, _From, S) ->
    {reply, image:resize(Img, 800, 600), S}.
```

Every resize in the system goes through one process, so a 64-core machine resizes one image at a time. CPU sits at 1/64 while requests time out. Fix: do the work in the caller or in a pool of workers; keep the server for coordination only.

### 4.4 A long NIF on a normal scheduler

```
static ERL_NIF_TERM hash_file(ErlNifEnv* env, int argc, const ERL_NIF_TERM argv[]) {
    /* reads and hashes a 2 GB file */
}
static ErlNifFunc funcs[] = {"hash_file", 1, hash_file, 0};
```

This runs for seconds on a normal scheduler. Every process queued there stalls, including heartbeats and timeouts, so unrelated requests suddenly miss their deadlines. Fix: flag it `ERL_NIF_DIRTY_JOB_IO_BOUND` (it's mostly I/O), or do file work in Erlang and hash chunks in a short NIF.

### 4.5 CPU work flagged as dirty I/O

```
{"matrix_mul", 2, matrix_mul, ERL_NIF_DIRTY_JOB_IO_BOUND}
```

It works, but there are 10 dirty I/O schedulers by default, typically more than you have cores. Filling them with CPU work competes with the normal schedulers for the cores, and the docs warn this “might starve ordinary schedulers.” Fix: `ERL_NIF_DIRTY_JOB_CPU_BOUND` for anything that computes.

## 4.6 Reading scheduler spin as load

```
container CPU 95% → autoscaler adds replicas
```

When schedulers run out of work, they briefly busy-wait before sleeping, to pick up new work faster. The OS counts that as CPU use, so a mostly idle node can look busy, and autoscaling on OS CPU scales the wrong way. Fix: measure real scheduler use with `scheduler:utilization/1` (`runtime_tools`). If you share CPUs with other workloads, consider turning busy-waiting down with the `+sbwt` flags.

## 5. Review checklist and sources

---

- ☐ Performance claims come from profiles or benchmarks of compiled code, not guesses or shell timings
- ☐ Benchmarks run for seconds in fresh processes
- ☐ CPU-heavy work is spread across processes, not funnelled through one server
- ☐ Every NIF returns within about 1 ms or runs on the correct dirty scheduler
- ☐ CPU-bound native work is flagged dirty CPU, not dirty I/O
- ☐ Capacity decisions use scheduler utilisation, not only OS CPU

### Sources

- Profiling, Efficiency Guide: <https://www.erlang.org/doc/system/profiling.html>
- Benchmarking, Efficiency Guide: <https://www.erlang.org/doc/system/benchmarking.html>
- `erl_nif` reference: [https://www.erlang.org/doc/apps/erts/erl\\_nif.html](https://www.erlang.org/doc/apps/erts/erl_nif.html)
- `erl` command (scheduler flags): [https://www.erlang.org/doc/apps/erts/erl\\_cmd.html](https://www.erlang.org/doc/apps/erts/erl_cmd.html)